

Cloud, Réseaux Virtuels et Conteneurisation

UE CRV — Sorbonne Université

Naceur.Malouch@lip6.fr

Cours 2 : Support noyau et utilitaires – Conteneurisation sous Linux

Mars 2026

Table des matières

1	Introduction à la Conteneurisation	2
1.1	Rappel : Types de virtualisation	2
1.2	Rappel : Hyperviseur (VMM)	2
1.3	Avantages et inconvénients des conteneurs	2
1.4	Conteneurisation dans le Cloud	2
1.5	Réseaux de conteneurs	3
2	FreeBSD vnet jail – Précurseur	3
3	Conteneurisation sous Linux : Namespaces	3
3.1	Définition et principes	3
3.2	clone() et les flags CLONE_*	3
3.2.1	CLONE_* (partage, pas de conteneur)	3
3.2.2	CLONE_NEW* (création de namespace = conteneur)	4
3.3	unshare() – Créer un namespace sans fork	4
3.3.1	Implantation de l'utilitaire unshare	4
3.4	Implémentation des namespaces dans le noyau	5
3.4.1	Principe général	5
3.4.2	Structure nsproxy	5
3.4.3	Champ gen – Extension dynamique	6
3.4.4	Principaux fichiers sources	6
3.4.5	Chaîne d'appels noyau pour unshare()	6
3.4.6	Numéros inode et /proc	6
3.5	setns() – Rejoindre un namespace existant	7
3.5.1	Utilitaire nsenter	7
4	Utilitaires iproute2 : ip netns	7
4.1	Fonctionnalités de ip netns	7
4.2	Exemple complet : deux conteneurs en communication	7
4.3	Vérification et test de connectivité	8
4.4	Créer un routeur	8
4.5	Créer un bridge	9
5	Cas d'usage avancés : pid_namespaces + mount	9
5.1	pid namespaces	9
5.2	Fichiers de configuration non partagés	9
6	Comparaison FreeBSD Jails vs Linux Namespaces	10
7	Sources	10

Introduction à la Conteneurisation

Rappel : Types de virtualisation

- **Virtualisation complète** : le logiciel de virtualisation assure à un autre système entier un accès complet aux ressources matérielles. *Exemples : VMWare Player, QEMU/KVM.*
- **Para-virtualisation** : le système est modifié pour interagir proprement avec le logiciel de virtualisation. *Exemples : XEN, ESX VSI.*
- **Virtualisation légère (au niveau SE)** : un ensemble de processus ont un accès isolé au système de fichiers, processeurs, mémoire, périphériques, etc. Certains objets du noyau peuvent être dupliqués. *Exemples : FreeBSD Jails, LXC.*

Le système virtualisé s'appelle aussi : **machine virtuelle**, image virtuelle, système invité (*guest OS*).

Rappel : Hyperviseur (VMM)

- **Type 1** : s'exécute directement sur le matériel (bare metal). → para-virtualisation.
- **Type 2** : s'exécute au sein du système hôte réel. → virtualisation complète.
- **Virtualisation au niveau SE (conteneurs) : pas de moniteur**. C'est le **noyau du système hôte** qui isole les processus et gère l'accès aux ressources.

Avantages et inconvénients des conteneurs

Avantages :

- Meilleure performance
- Nombre très grand de conteneurs sur la même machine
- Création et dimensionnement à la demande
- Dynamisme, rapidité et flexibilité

Inconvénients :

- Certains services sont plus difficiles à mettre en place dans des conteneurs
- Mise à jour continue du noyau du système hôte requise
- Nécessite plus de mécanismes de sécurisation

Conteneurisation dans le Cloud

Trois configurations possibles :

- Conteneurs **dans** des VMs
- Conteneurs **sur** système hôte directement
- VMs sans conteneurs (cas classique)

Dans le modèle CaaS, la couche "Conteneurs" est fournie par le prestataire, au-dessus de la "Virtualisation / Hyperviseur".

Réseaux de conteneurs

Pour créer des réseaux de conteneurs, il faut :

1. Créer plusieurs conteneurs sur un ou plusieurs systèmes
2. Activer les fonctionnalités réseaux (ex : acheminement des paquets)
3. **Virtualiser la pile protocolaire** (*Network Stack Virtualization*) → duplication de la pile protocolaire
4. Créer des interfaces virtuelles rattachées aux piles + liens virtuels entre les piles

Principe clé : **Plusieurs états du noyau opèrent sur une seule instance du noyau**. Chaque conteneur a sa propre pile protocolaire (IRV) reliée à l'instance principale (IR).

Applications possibles : gestion distante des conteneurs, Cluster Computing, VPC, etc.

FreeBSD vnet jail – Précurseur

- **jail** : emprisonne des processus dans un espace d'exécution restreint
 - **vnet jail** : attribue à l'environnement jail une pile protocolaire propre (adresses, table de routage, etc.)
 - Technologie complète via la commande `jail`
- L'équivalent Linux s'appelle **namespace** (espace de nom).

Conteneurisation sous Linux : Namespaces

Définition et principes

Namespace – Espace de nom

Mécanisme du noyau Linux permettant l'exécution de processus dans un environnement **isolé et/ou dupliqué**. Permet notamment de dupliquer la pile protocolaire réseau.

Principaux appels système : `clone()`, `unshare()`, `setns()`

```
man namespaces
```

`clone()` et les flags `CLONE_*`

`CLONE_*` (partage, pas de conteneur)

```
clone(..., CLONE_*, ...)
```

- Crée un nouveau processus
- Les flags `CLONE_*` permettent de **partager** certaines variables entre père et fils (au lieu de deux copies indépendantes)

- Exemple : `CLONE_FILES` → partager la même table des descripteurs de fichiers
- Ces flags ne permettent PAS la création de conteneurs

```
#define CLONE_FILES 0x00000400
```

`CLONE_NEW*` (création de namespace = conteneur)

```
clone(..., CLONE_NEW*, ...)
```

- Crée un nouveau processus
- Les flags `CLONE_NEW*` permettent de créer un **nouveau namespace** pour le nouveau processus

Flag	Namespace créé
<code>CLONE_NEWNET</code>	Network devices, stack → Network Stack Virtualization
<code>CLONE_NEWPID</code>	Process IDs
<code>CLONE_NEWNS</code>	Mount points
<code>CLONE_NEWUSER</code>	User and group IDs
<code>CLONE_NEWUTS</code>	Hostname, NIS domain name
<code>CLONE_NEWIPC</code>	IPC resources

Combinaison possible :

```
CLONE_NEWNET | CLONE_NEWNS
```

`unshare()` – Créer un namespace sans fork

```
unshare(CLONE_NEW*)
```

- NE crée PAS un nouveau processus
- Crée un nouveau namespace pour le **processus appelant lui-même**

Exemples d'utilisation :

```
unshare(CLONE_NEWNET);
unshare(CLONE_NEWNET | CLONE_NEWNS);
```

Implantation de l'utilitaire `unshare`

L'utilitaire **unshare** (en ligne de commande) est basé sur l'appel `unshare()`. Version simplifiée :

```
main(int argc, char *argv[])
{
    int flags, opt;
    flags = 0;

    while ((opt = getopt(argc, argv, "imnpU")) != -1) {
        switch (opt) {
```

```

        case 'i': flags |= CLONE_NEWIPC;    break;
        case 'm': flags |= CLONE_NEWNS;    break;
        case 'n': flags |= CLONE_NEWNET;   break;
        /* case 'p': flags |= CLONE_NEWPID; break; */
        case 'u': flags |= CLONE_NEWUTS;   break;
        case 'U': flags |= CLONE_NEWUSER;  break;
    }
}
unshare(flags);
execvp(argv[optind], &argv[optind]);
}

```

L'implantation complète se trouve dans le package **util-linux** : `sys-utils/unshare.c`

Permet d'exécuter un programme dans un ou plusieurs nouveaux namespaces, avec possibilité de les rendre **persistants**.

Implémentation des namespaces dans le noyau

Principe général

- Chaque namespace est géré par une **liste chaînée** de structures
- Chaque structure représente un namespace et contient ses variables
- Le support de la conteneurisation consiste à **intégrer dans la structure du namespace** certaines variables auparavant déclarées globalement

Structure nsproxy

Une structure **nsproxy** fait le lien entre la structure de processus (`task_struct`) et ses namespaces :

```

/* Fichier : include/linux/nsproxy.h */
struct nsproxy {
    atomic_t count;
    struct uts_namespace    *uts_ns;
    struct ipc_namespace    *ipc_ns;
    struct mnt_namespace    *mnt_ns;
    struct pid_namespace    *pid_ns_for_children;
    struct net               *net_ns;
    struct cgroup_namespace *cgroup_ns;
};
extern struct nsproxy init_nsproxy;

```

La chaîne de pointeurs est :

`task_struct` → `nsproxy` → `net_ns (net)` → ...

La liste de tous les net namespaces s'appelle `net_namespace_list`. Le premier net namespace est `init_net`.

Les structures **sock** (sockets) et **net_device** (interfaces) pointent aussi vers le net adéquat.

Champ `gen` – Extension dynamique

Le champ `gen` dans la structure `net` permet de référencer des structures supplémentaires sans encombrer la structure `net` :

- `net_generic(net, id)` : accède à la structure dupliquée d'un `net` donné
- `register_pernet_subsys()` / `unregister_pernet_subsys()` : ajoute/supprime des opérations d'initialisation et de terminaison d'un sous-système réseau par `net`
- `register_pernet_device()` / `unregister_pernet_device()` : idem pour les interfaces réseaux

Principaux fichiers sources

```
kernel/fork.c                # clone(), unshare() syscalls
include/linux/nsproxy.h
kernel/nsproxy.c
include/net/net_namespace.h
net/core/net_namespace.c
include/net/netns/ipv4.h
include/net/netns/generic.h
```

Chaîne d'appels noyau pour `unshare()`

1. `unshare()`
2. `unshare_nsproxy_namespaces()`
3. `create_new_namespaces()`
4. `create_nsproxy()`
5. `copy_net_ns()`
6. `net_alloc()` / `setup_net()`
7. `switch_task_namespaces()`

Numéros inode et `/proc`

- Chaque namespace a un **numéro inode unique** dans le pseudo-système de fichiers `/proc` (*unique proc inode number*)
- **Les namespaces n'ont pas de noms** nativement
- Référencement depuis l'espace utilisateur via `/proc/pid/ns/*`

```
$ ls -al /proc/2454/ns/
lrwxrwxrwx 1 root root 0 Apr 8 09:12 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 Apr 8 09:12 mnt -> mnt:[4026531840]
lrwxrwxrwx 1 root root 0 Apr 8 09:12 net -> net:[4026531956]
lrwxrwxrwx 1 root root 0 Apr 8 09:12 pid -> pid:[4026531836]
lrwxrwxrwx 1 root root 0 Apr 8 09:12 uts -> uts:[4026531838]
```

Pour attribuer un nom, on effectue un **bind mount** :

```
mount --bind /proc/2454/ns/net /un/chemin/quelconque
# Une convention : /root/mycontainers/PC1/net
```

setns() – Rejoindre un namespace existant

```
setns(fd, CLONE_NEW*)
```

- Permet au processus appelant de **rejoindre** le namespace d'un autre processus
- CLONE_NEW* spécifie le type du namespace (optionnel, 0 = type quelconque)
- fd : descripteur de fichier obtenu en ouvrant un fichier /proc/pid/ns/*

Exemple :

```
setns(fd, CLONE_NEWNET);
execvp(argv[1], &argv[1]);
```

Deux comportements :

- Si le processus entre dans **TOUS** les namespaces du processus cible → le nsproxy de la task_struct pointe directement vers le nsproxy de la cible.
- Si le processus entre dans **un ou plusieurs** namespaces → appel de create_new_namespaces() et create_nsproxy() pour créer un nouveau proxy hybride.

Utilitaire nsenter

Basé sur setns(), disponible dans le package **util-linux** (sys-utils/nsenter.c).

Exemples :

```
nsenter --net=/root/mycontainers/PC1/net MyAppli --fconfig /home/do/mycfile
nsenter --net=/proc/2454/ns/net MyAppli --fconfig /home/do/mycfile
```

Permet d'exécuter un programme dans un ou plusieurs **namespaces existants**.

Utilitaires iproute2 : ip netns

Fonctionnalités de ip netns

L'utilitaire **ip** de iproute2 a été étendu pour les namespaces réseaux :

Commande	Action
ip netns add/del/list	Créer/supprimer/lister des net namespaces nommés
ip link ... veth ...	Créer des paires d'interfaces Ethernet virtuelles
ip link ... netns ...	Associer une interface à un namespace réseau
ip netns exec NS cmd	Exécuter une commande dans un namespace donné

Basé essentiellement sur unshare(), setns() et mount() (bind mounts).

Les namespaces sont enregistrés par bind mount dans /var/run/netns/.

Exemple complet : deux conteneurs en communication


```
# Creer deux net namespaces
root@linux# ip netns add netpc1
root@linux# ip netns add netpc2
root@linux# ip netns list
netpc2
netpc1

# Creer une paire d'interfaces veth (lien virtuel)
root@linux# ip link add veth1 type veth peer name veth2

# Associer chaque interface a son namespace
root@linux# ip link set veth1 netns netpc1
root@linux# ip link set veth2 netns netpc2

# Configurer les adresses IP
root@linux# ip netns exec netpc1 ip addr add 10.0.0.1/24 dev veth1
root@linux# ip netns exec netpc2 ip addr add 10.0.0.2/24 dev veth2

# Activer les interfaces
root@linux# ip netns exec netpc1 ip link set dev veth1 up
root@linux# ip netns exec netpc2 ip link set dev veth2 up
```

Vérification et test de connectivité

```
root@linux# ip netns exec netpc1 ip addr show
5: veth1: <...UP...> mtu 1500 ...
    inet 10.0.0.1/24 scope global veth1

root@linux# ip netns exec netpc2 ip addr show
4: veth2: <...UP...> mtu 1500 ...
    inet 10.0.0.2/24 scope global veth2

root@linux# ip netns exec netpc1 ping 10.0.0.2
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.151 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.044 ms
```

Le lien veth peut être configuré comme un lien ordinaire :

```
# Exemple : appliquer un qdisc (QoS, limitation de debit, etc.)
root@linux# ip netns exec netpc1 tc qdisc add dev veth1 root ...
```

Créer un routeur

```
root@linux# ip netns exec Router \
    sysctl net.ipv4.conf.vethr1.forwarding=1
root@linux# ip netns exec Router \
    sysctl net.ipv4.conf.vethr2.forwarding=1
# + processus de routage ou routage statique
```

Créer un bridge

```
root@linux# ip link add name mybridge type bridge
root@linux# ip link set mybridge up
root@linux# ip link set vethb1 master mybridge
root@linux# ip link set vethb2 master mybridge
```

Cas d'usage avancés : pid_namespaces + mount

pid namespaces

Problème sans pid namespace

Si deux processus A et B tournent dans deux namespaces réseau différents mais le **même pid namespace**, leurs PIDs sont uniques globalement mais visibles depuis les deux espaces.

Avec des pid namespaces distincts :

```
# Dans netpc2
root@netpc2# unshare --fork --pid --mount-proc A
root@netpc2# pgrep -fn A
3401
root@netpc2# nsenter -m -p -t 3401 B
# Maintenant le pid de A est 1, le pid de B est e.g. 160
# dans le nouveau espace de noms /proc/3401/ns/pid
```

Alternative avec namespaces persistants :

```
root@netpc1# unshare --fork --pid --mount=/myc/pc1/mnt A
root@netpc1# nsenter --pid=/myc/pc1/pid \
                  --mount=/myc/pc1/mnt B
root@netpc1# nsenter --pid=/myc/pc1/pid \
                  --mount=/myc/pc1/mnt mount -t proc proc /proc
```

Fichiers de configuration non partagés

Les fichiers placés dans `/etc/netns/{nom_ns}/` sont automatiquement montés dans `/etc/` lors de l'exécution d'une commande via `ip netns exec` :

```
root@linux# ip netns exec netpc1 cat /etc/resolv.conf
nameserver 193.122.54.200
root@linux# cat /etc/netns/netpc1/resolv.conf
nameserver 193.122.54.200

root@linux# ip netns exec netpc2 cat /etc/resolv.conf
nameserver 80.167.98.1
```

Pour des fichiers arbitraires, utiliser bind mount :

```
root@linux# nsenter -m -t 3324 mount --bind \
                  /myc/noshare/netpc1/appli.conf /root/appli.conf
```

```
root@linux# nsenter -m -t 3401 mount --bind \
    /myc/noshare/netpc2/appli.conf /root/appli.conf
```

Comparaison FreeBSD Jails vs Linux Namespaces

Critère	Jails (FreeBSD)	Namespaces (Linux)
Appels systèmes	Oui	Oui
Utilitaire officiel	Oui (complet) : jail	unshare, nsenter, ip netns
Logiciels haut niveau	Ezjail, CBSD, etc.	LXC, Docker, etc.
Réseau virtualisé	VNET jails	Network Namespaces
Option noyau	VIMAGE	CONFIG_NET_NS
Structures internes	Liste chaînée de vnet	Liste chaînée de net
Fichiers noyau	vnet.h, vnet.c	net_namespace.h, net_namespace.c
Config réseau	ifconfig	ip
Interfaces virtuelles	Oui	Oui

Sources

- `man namespaces` – Manuel Linux
- `man clone`, `man unshare`, `man setns` – Manuel Linux
- Noyau Linux – `kernel/fork.c`, `kernel/nsproxy.c`, `net/core/net_namespace.c`
- Package `util-linux` – `sys-utils/unshare.c`, `sys-utils/nsenter.c`
- Copyright Naceur.Malouch@lip6.fr – All rights reserved.